САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Смирнова Карина

К3441

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.JS.

Каким требованиям ваше приложение должно соответствовать:

- Должен быть подключён роутер
- Должна быть реализована работа с внешним API (желательно посредством axios)
- Разумное деление на компоненты (продемонстрируйте понимание компонентного подхода)
- Использование composable для выделения повторяющегося функционала в отдельные файлы

Ход работы

Сначала был создан новый проект на основе Vue 3 и Vite. В проект были подключены основные библиотеки, необходимые для работы приложения: Vue Router для переходов между страницами, Pinia для хранения общего состояния и Axios для выполнения HTTP-запросов. Также были добавлены команды для одновременного запуска клиентской части и JSON Server Auth. Настройка Vite была дополнена перенаправлением запросов с адреса /api на локальный сервер, работающий на отдельном порту.

После этого в проект были перенесены данные и графические материалы из предыдущей версии RentSpace. В файле db.json хранятся пользователи, объявления, бронирования, чаты и сообщения, а в routes.json задаются правила доступа к соответствующим ресурсам. Изображения объектов, фоновые изображения и SVG-спрайт были размещены в каталоге публичных ресурсов, благодаря чему они доступны как обычные статические файлы.

Затем была разработана общая структура одностраничного приложения. В файле main.js создаётся экземпляр Vue, подключаются Pinia и Vue Router, после чего приложение монтируется в корневой элемент страницы. Компонент App.vue содержит область для отображения текущего маршрута и глобальный компонент уведомлений. Основное содержимое страницы формируется с помощью представлений и

layout-компонентов, поэтому корневой компонент не содержит лишней разметки.

Для страниц входа и регистрации был создан отдельный layout без основной навигации. Для остальных страниц используется общий layout, который включает header, основное содержимое и footer. Header содержит логотип сервиса, ссылки на основные разделы, переключатель темы и меню пользователя. Если пользователь не авторизован, вместо личного меню отображаются ссылки для входа и регистрации.

После создания общего каркаса была настроена маршрутизация. Для входа, регистрации, поиска, просмотра объекта, личного кабинета и раздела сообщений были созданы отдельные маршруты. В метаданных маршрутов указываются заголовки страниц и требования к авторизации. Перед каждым переходом выполняется проверка состояния пользователя: защищённые страницы доступны только после входа, а уже авторизованный пользователь не может повторно открыть страницы входа и регистрации. При перенаправлении на форму входа сохраняется адрес исходной страницы, чтобы после успешной авторизации пользователь мог вернуться к ней.

Далее было реализовано глобальное состояние авторизации с помощью Pinia. Store хранит данные текущего пользователя, токен и вычисляемый признак авторизации. При успешном входе данные сохраняются как в реактивном состоянии приложения, так и в локальном хранилище браузера. Благодаря этому пользователь остается авторизованным после обновления страницы, а изменения профиля сразу отображаются в header и других компонентах.

Для работы с сервером был создан отдельный API-слой. Общий Axios-клиент отвечает за базовый адрес запросов и автоматически добавляет токен авторизации. Остальные API-файлы разделены по назначению: авторизация, пользователи, объявления, бронирования, чаты и сообщения. Такое разделение позволило не размещать детали HTTP-запросов внутри компонентов интерфейса.

После настройки общей архитектуры были разработаны страницы входа и регистрации. Для них создана общая оболочка с фоновым изображением, логотипом и формой. Поле пароля вынесено в отдельный

компонент и поддерживает показ и скрытие введенных символов. При регистрации проверяется корректность введенных данных и совпадение паролей, а после успешного создания аккаунта открывается модальное окно с переходом на страницу входа.

Затем была реализована страница поиска недвижимости. Форма фильтров позволяет выбрать способ аренды, расположение, тип объекта и диапазон стоимости. Выбранные значения передаются в представление, которое запрашивает подходящие объявления с сервера. Полученные данные отображаются в виде отдельных карточек, а пользователь может изменить порядок сортировки или сбросить фильтры. Если подходящих объектов нет, вместо списка показывается соответствующее сообщение.

Для страницы отдельного объекта были созданы компоненты галереи и бронирования. Галерея отображает основное изображение и дополнительные фотографии, а выбранную фотографию можно открыть в модальном окне. В зависимости от типа аренды пользователь видит либо форму бронирования, либо варианты связи с владельцем. Для посуточной аренды автоматически рассчитываются продолжительность проживания и итоговая стоимость. Если гость пытается отправить заявку или открыть чат, приложение предлагает ему сначала войти в аккаунт.

После этого был разработан личный кабинет. В нем отображаются данные пользователя, статистика, список аренд и собственные объявления. Информация разделена на вкладки, между которыми можно переключаться как мышью, так и клавиатурой. Пользователь может изменить имя, электронную почту и телефон, а также скрыть или повторно опубликовать своё объявление. После сохранения обновленные данные передаются на сервер и записываются в общее состояние приложения.

На следующем этапе был реализован раздел сообщений и сделок. В нем пользователь может просматривать список чатов, выбирать собеседника и отправлять новые сообщения. История выбранного чата загружается с сервера, а после отправки сообщения обновляется краткая информация в списке чатов. Во вкладке сделок отображаются бронирования, связанные с текущим пользователем. Сделки можно фильтровать по статусу, открывать связанный объект, переходить в соответствующий чат и отменять заявки, которые еще ожидают подтверждения.

Повторяющаяся реактивная логика была вынесена в composable-функции. Отдельные composables отвечают за переключение темы, показ уведомлений и управление вкладками. Форматирование цен, дат, времени, статусов и инициалов пользователя размещено в модуле вспомогательных функций. Это позволило сократить количество повторяющегося кода и упростить компоненты.

В завершение были перенесены и доработаны стили приложения. Общие цвета, размеры и параметры светлой и тёмной темы задаются в глобальных таблицах стилей, а оформление конкретных элементов размещено внутри соответствующих Vue-компонентов. Для адаптации интерфейса к экранам разной ширины используются медиазапросы. Интерактивные элементы, такие как модальные окна, вкладки, меню и галерея, управляются средствами Vue без подключения JavaScript-части Bootstrap.

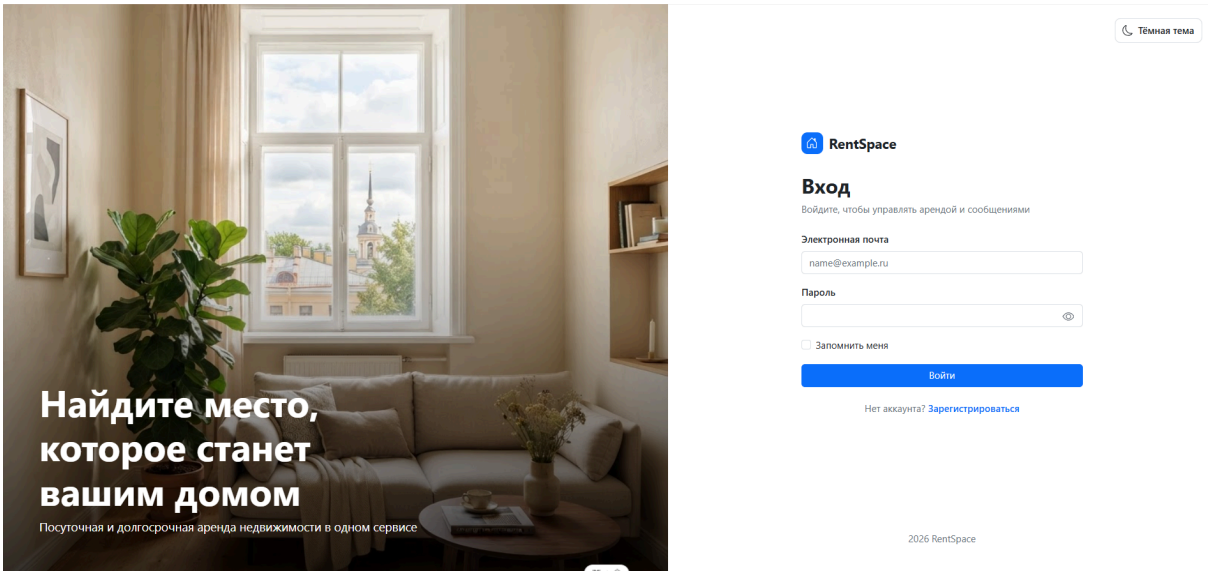


Рисунок 1 - Страница входа

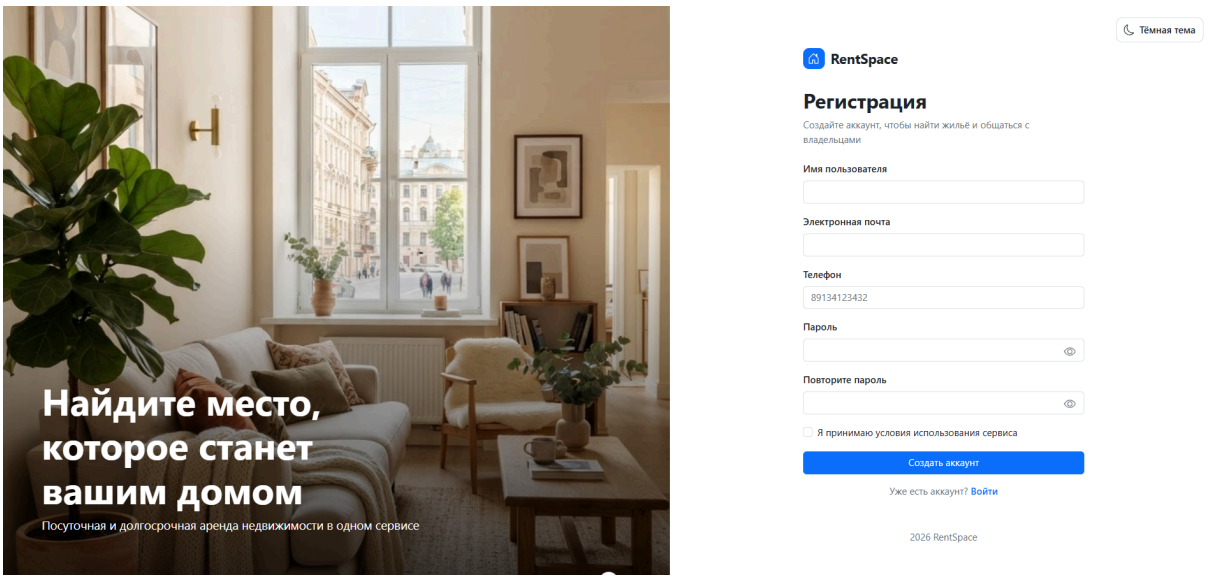


Рисунок 2 - Страница регистрации

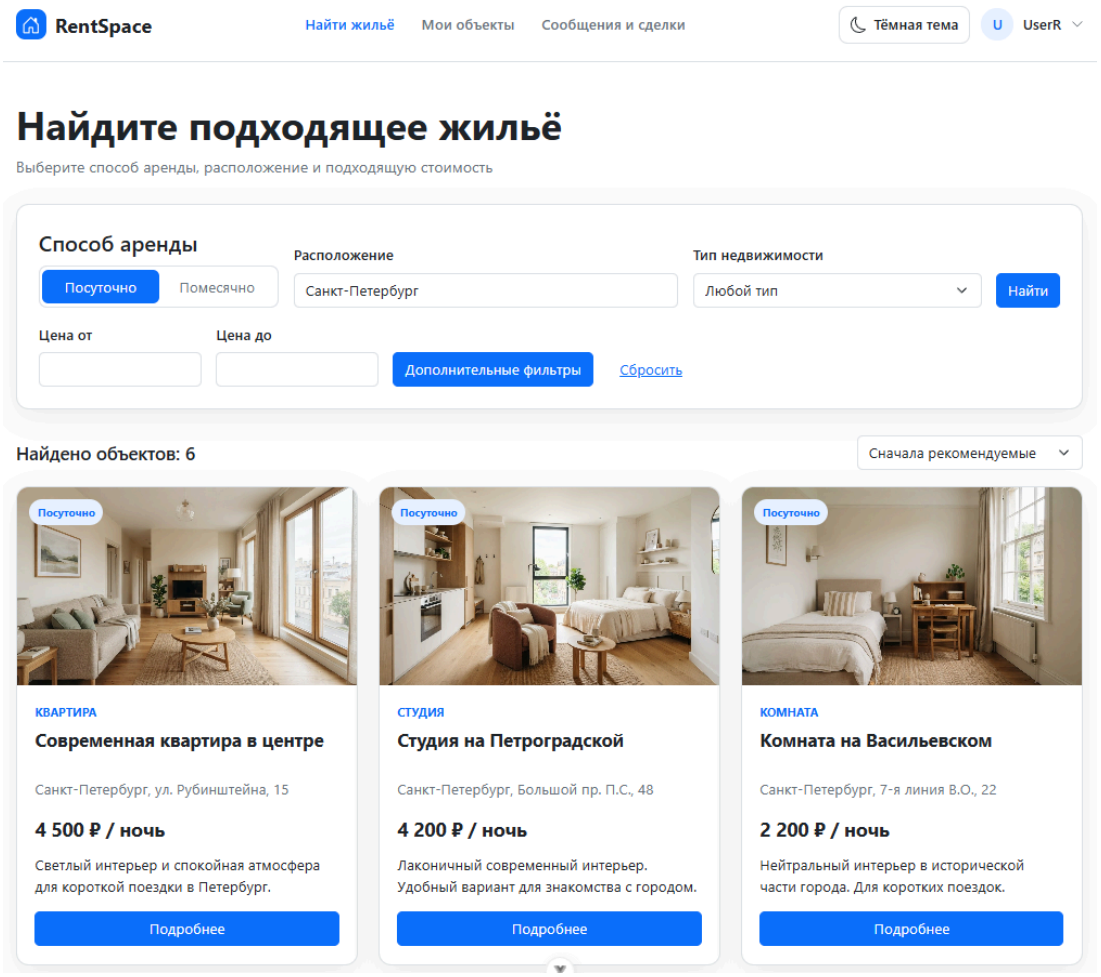


Рисунок 3 - Страница поиска

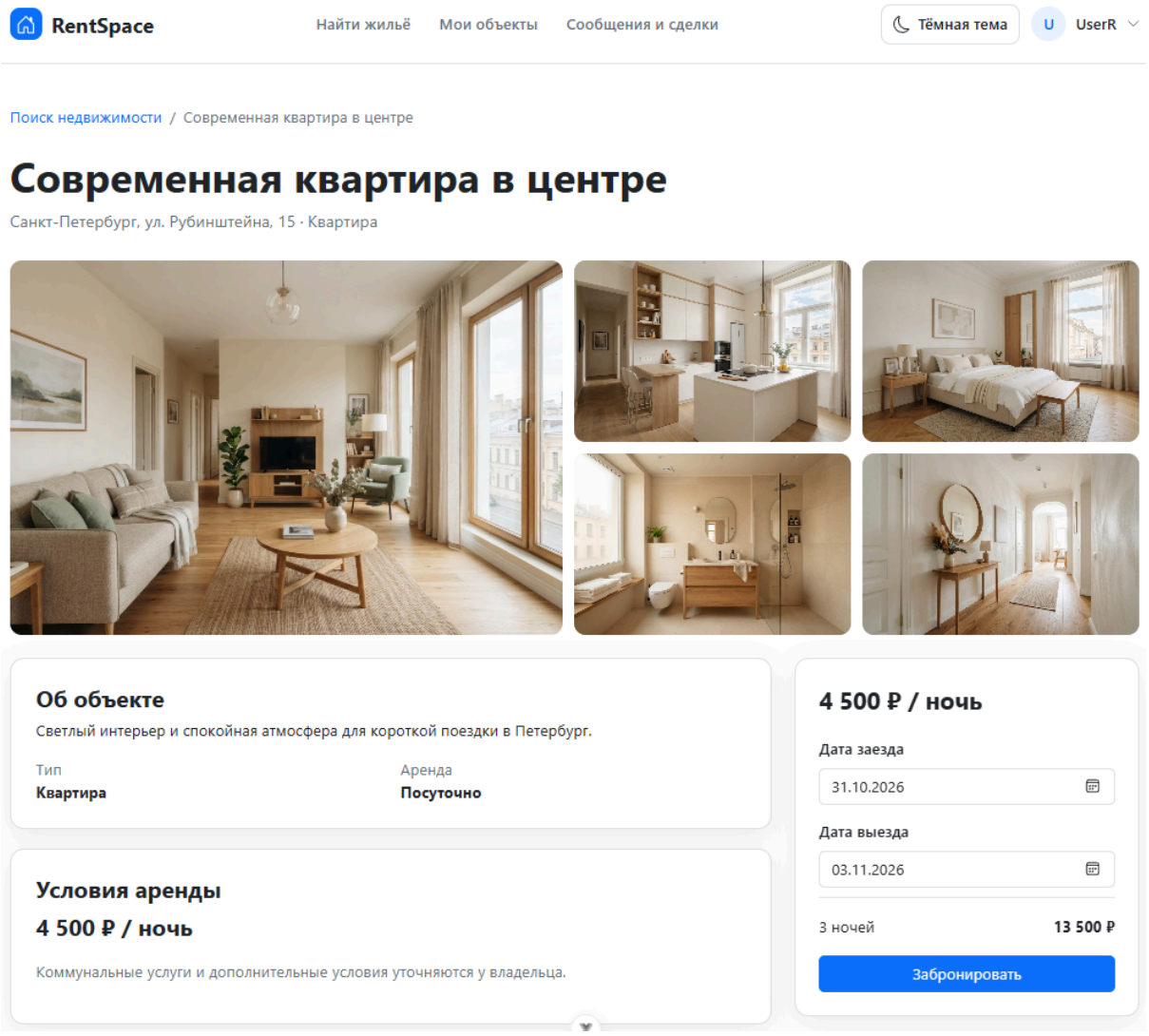


Рисунок 4 - Страница объекта

Вывод

В ходе выполнения лабораторной работы я изучила процесс переноса многостраничного веб-приложения на Vue и построения на его основе SPA. Я разобралась в назначении представлений, компонентов, layout-компонентов, props и пользовательских событий. Также была освоена маршрутизация с использованием метаданных и проверок доступа к защищенным страницам. В результате исходный проект RentSpace был разделен на независимые и переиспользуемые части.

Отдельное внимание было уделено организации состояния и взаимодействию с сервером. С помощью Pinia была реализована общая авторизация, composable-функции позволили вынести повторяемую реактивную логику, а отдельный API-слой упростил выполнение запросов. Компонентный подход сделал структуру приложения более понятной и удобной для дальнейшего расширения. Итоговое приложение сохраняет возможности предыдущей версии, но использует реактивное обновление интерфейса и переходы между страницами без полной перезагрузки.